Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”; MT circulara
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

O MT poate fi descrisa in 3 moduri:

- formal:

  sunt date complet Q, Σ, Γ, δ;

- la nivelul implementarii MT:

  se utilizeaza limba naturala pt a defini:

  - modul in care se deplaseaza cursorul,
  - modul de memorare a informatiei de pe banda de lucru;

- la nivelul cel mai inalt:

  se utilizeaza limba naturala pt a descrie un algoritm, ignorand complet modul de implementare a acestuia.

Variante de masini Turing

Definim formatul si notatia utilizate pt a descrie o MT:

datele de intrare constau intotdeauna dintr-o secventa de simboluri;

daca intrarea trebuie sa fie un obiect (un graf, un polinom, un automat etc. sau o combinatie a acestora), atunci va trebui mai intai sa-l reprezentam printr-o secventa;

Intrucat codificarile alternative pot fi decodificate unele in altele de catre MT (eficient) => putem alege orice codificare pt obiectul de intrare;

Vom nota:

- 1! obiect O, codificat printr-o secventa, prin <O>,
- mai multe obiecte $O_1,O_2,..,O_n$, codificate printr-o unica secventa, prin < $O_1,O_2,..,O_n$ >.

Variante de masini Turing

- prima linie a textului va descrie datele de intrare ale MT;
- daca intrarea este:
  - chiar o secventa de simboluri w => ea este interpretata ca un cuvant peste un alfabet si notata w:

    $\Sigma_{bool} = \{0, 1\}$,

    $\Sigma_{latin} = \{a,b,c,\ldots,z\}$,

    $\Sigma_{logic} = \{ 0,1,(,), \neg,\wedge,\vee,x \}$,

    $\Sigma_{pol} = \{ 0,1,(,), +,*,\hat{},x \}$ etc.;
  - codificarea unui obiect <A> => MT incepe automat cu VERIFICAREA CORECTITUDINII CODIFICARII.
- algoritmul este impartit in etape → pasi de calcul numerotati;
- bloc → indentare.

Variante de masini Turing

Problema:

Sa se gaseasca un algoritm care sa verifice daca un graf neorientat oarecare este conex sau nu

Formalizare:

Fie A = un limbaj care consta din toate secventele care reprezinta grafuri neorientate conexe =>

A = {<G> | G este un graf neorientat, conex};

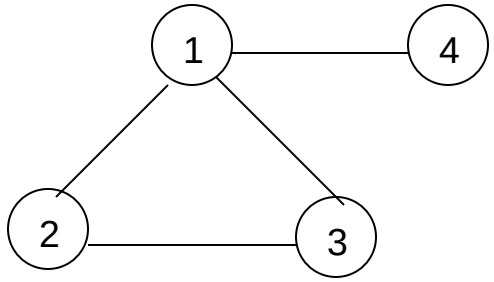
trebuie sa gasim o MT decidenta care sa decida asupra limbajului A

M = “Fie intrarea <G> (adica: o codificare a grafului G):

1. Se verifica intai corectitudinea codificarii <G>; daca G nu este corect codificat, M respinge
2. Se selecteaza “primul” nod din G si se marcheaza
3. Se reia pasul urmator pana cand nu se mai pot marca noi noduri:
   4. Fie un nod oarecare v din G; daca el este legat printr-o muchie de un nod deja marcat, atunci nodul v trebuie marcat si el
5. Se scaneaza toate nodurile din G pt a verifica daca sunt toate marcate sau nu. Daca sunt marcate toate nodurile, atunci M accepta; altfel, M respinge.”

Variante de masini Turing

Exemplu



<G>=((1,2,3,4);(1,2),(1,3),(1,4),(2,3))

Explicam modul de codificare a lui G:

MT incepe prin a verifica corectitudinea codificarii <G>

Daca testul se incheie cu succes (codificarea e corecta), atunci MT trece la prima etapa a algoritmului:

1. Se selecteaza “primul” nod din G si se marcheaza
2. Se reia pasul urmator pana cand nu se mai pot marca noi noduri
3. Fie un nod oarecare v din G; daca el este legat printr-o muchie de un nod deja marcat, atunci nodul v trebuie marcat si el
4. Se scaneaza toate nodurile din G pt a verifica daca sunt toate marcate sau nu. Daca sunt marcate toate nodurile, atunci M accepta; altfel, M respinge.”

Variante de Masini Turing

Codificari; exemple (numere naturale, grafuri, retele)

1. Codificarea binara a numarului natural $x \in N$ este cuvantul

$u=u_n u_{n-1} \ldots u_2 u_1 \in \Sigma_{bool}^n$, $u_i \in \Sigma_{bool}$, $\forall 1 \le i \le n$, adica:

$$x = Nr(u) = \sum_{i=1}^{n} u_i \cdot 2^{i-1}$$

De exemplu: $Nr(1011)=1 \cdot 2^0+1 \cdot 2^1+0 \cdot 2^2+1 \cdot 2^3=11$

O alta codificare a numerelor naturale: $\Sigma=\{1\} \Rightarrow 0 \rightarrow 1,\ 1 \rightarrow 11,\ 2 \rightarrow 111$ .....

2. Fie G=(V,U) un graf cu n noduri si $M_G=(a_{ij})_{i,j=1,\ldots n}$ matricea sa de adiacenta. Daca utilizam alfabetul $\Sigma=\{0,1,\&\}$, atunci graful G poate fi codificat prin cuvantul

$$a_{11}a_{12}\ldots a_{1n}\&a_{21}a_{22}\ldots a_{2n}\&\ldots\&a_{n1}a_{n2}\ldots a_{nn}$$

2. Codificarea grafurilor prin cuvinte peste alfabetul Σ_{bool}.
3. Codificarea retelelor prin cuvinte peste alfabetul $\Sigma=\{0,1,\&\}$, ponderile fiind numere intregi pozitive.

Variante de Masini Turing

Codificari; exemple (formule logice).

4. X={x_1,x_2,x_3…} {$\neg,\wedge,\vee$}??

Solutia?

- utilizam alfabetul Σ_{logic} si
- codificam fiecare variabila x_j prin xbin(j) unde

  $x \in \Sigma_{logic}$

  bin(j) este cel mai scurt cuvant peste Σ_{bool} cu proprietatea Nr(bin(j))=j

De exemplu: formula

$$\Phi = (x_1 \vee \bar{x}_4 \vee x_7) \wedge (x_2 \vee \bar{x}_1) \wedge (x_4 \vee \bar{x}_8)$$

va fi reprezentata prin cuvantul

$$w_\Phi = (x1 \vee \neg(x100) \vee x111) \wedge (x10 \vee \neg(x1)) \wedge (x100 \wedge \neg(x1000))$$

Variante de Masini Turing

Codificari; exemple (polinoame).

5. Analog, utilizam alfabetul Σ_{pol} si codificam fiecare variabila x_j prin xbin(j)

$\Rightarrow$ polinomul

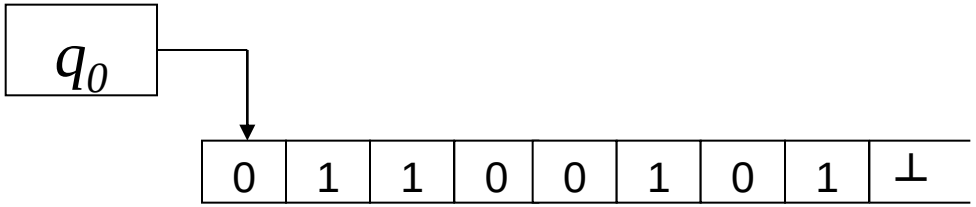
$$p(x_1, x_2, x_3) = (x_1 + 3x_2)^2 \cdot (2x_2 + 4x_3) \cdot x_1^2 =$$
$$= 36x_1^2x_2^2x_3 + 18x_1^2x_2^3 + 24x_1^3x_2x_3 + 12x_1^3x_2^2 + 2x_1^4x_2 + 4x_1^4x_3$$

va fi reprezentat prin cuvantul $w_p \in \Sigma_{pol}^*$

$$w_p = 100100 \cdot x1\text{^}10 \cdot x10\text{^}10 \cdot x11 + 10010 \cdot x1\text{^}10 \cdot x10\text{^}11 + \ldots + 100 \cdot x1\text{^}100 \cdot x101$$

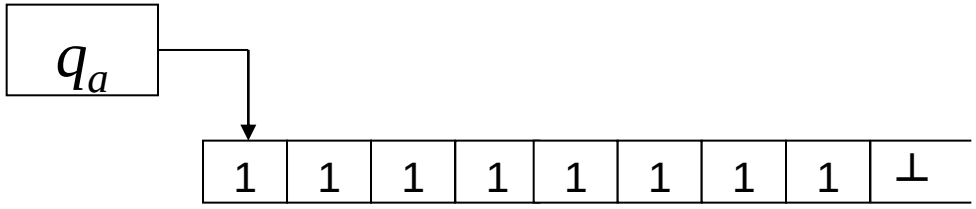
Variante de Masini Turing

Exemplul 1: Inlocuirea simbolurilor



M_1 = “Fie intrarea w∈{0,1}*:

1. Se scaneaza banda de intrare;
   daca se intalneste un simbol diferit de 0 sau 1, M_1 respinge
2. Se aduce cursorul in extremitatea stanga si se reia scanarea benzii
3. Atat timp cat simbolul citit NU este ⊥, se executa pasul P4
   4. Daca simbolul citit este 0, el se inlocuieste cu 1
5. M_1 accepta’’



Conventie: dupa prelucrarea celulei curente, cursorul se deplaseaza la stanga; daca e nevoie de o alta deplasare ea va fi specificata explicit, printr-un pas de sine statator.

Variante de Masini Turing

M_1' = "Fie intrarea w∈{0,1}*:

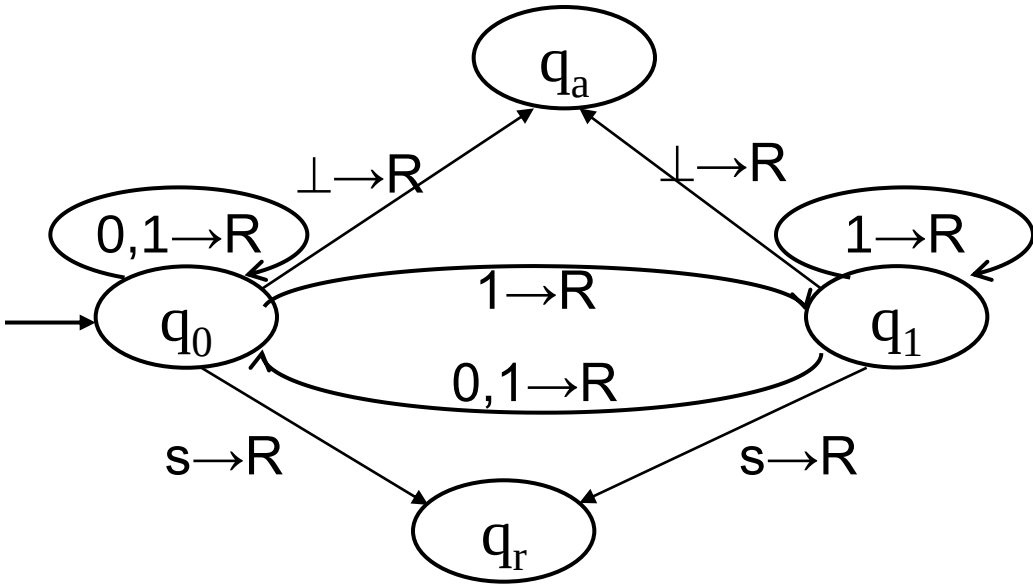
1. Se scaneaza banda de intrare; daca se intalneste un simbol diferit de 0 sau 1, M_1 respinge

2. Se aduce cursorul in extremitatea stanga si se reia scanarea benzii;

3. Daca simbolul curent este 0 se inlocuieste cu 1 şi cursorul se deplaseaza o pozitie la dreapta;

4. Daca simbolul curent este 1 cursorul se deplaseaza o pozitie la dreapta;

5. Daca simbolul curent este ⊥ M_1 accepta, altfel se reia de la P_3.



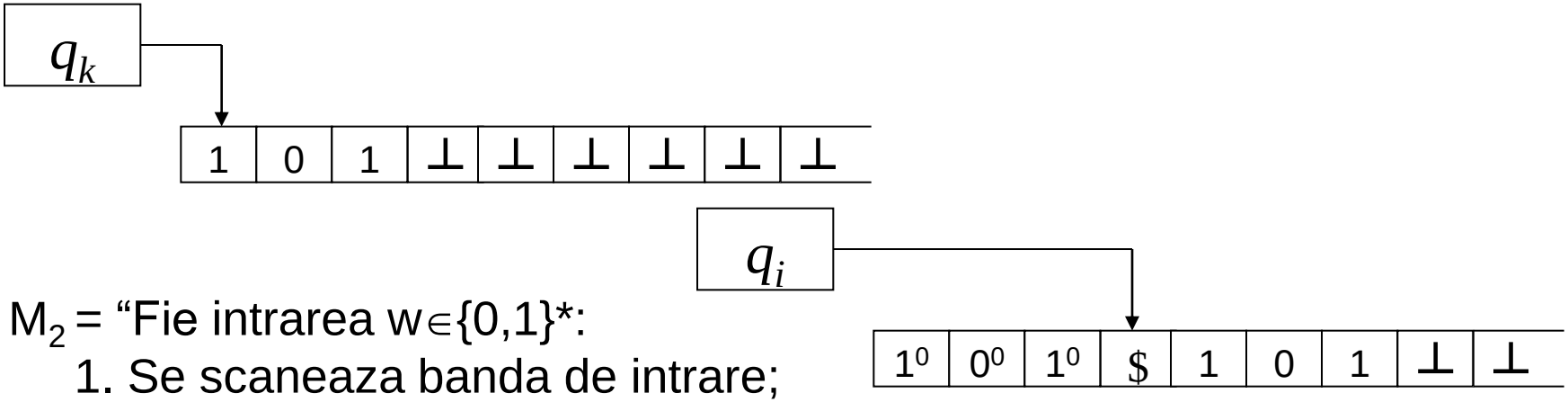
Variante de Masini Turing

Exemplul 2: Dublarea unui numar; Copierea unei secvente

f: N→N, f(x)=2x, ∀x∈N

g: Σ*→Σ*, g(u)=uu, ∀u∈Σ*, in particular: Σ={0,1} ⇒x=Nr(u);

ex: u=5=Nr(101) => g(101)= 101101

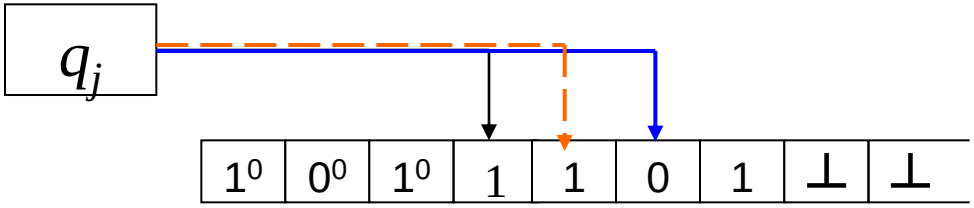


M_2 = "Fie intrarea w∈{0,1}*:

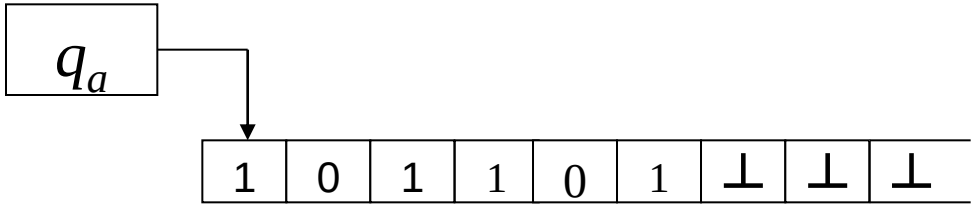
1. Se scaneaza banda de intrare;
   daca se intalneste un simbol diferit de 0 sau 1, M_1 respinge
2. Daca banda este vida, M_2 accepta (ε*ε= ε)
3. Se inlocuieste primul ⊥ (blank) cu simbolul special $ si se aduce cursorul in extremitatea stanga
4. Atat timp cat simbolul citit nu este $ si nu este marcat se copiaza in prima celula libera din dreapta simbolului $ si apoi se marcheaza cu un punct.

Variante de Masini Turing

5. Se muta cursorul o pozitie la dreapta
6. Atat timp cat simbolul citit nu este $\perp$ se executa pasii P7, P8:
   7. Se copiaza simbolul citit in celula din stanga cursorului
   8. Se deplaseaza cursorul cu 2 pozitii la dreapta



9. Se aduce cursorul in extremitatea stanga. Atat timp cat simbolul citit este marcat se executa pasul P10
   10. Se elimina marcajul
11. M_2 accepta.



Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”; MT circulara
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Exemplul 1

Fie MT = $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ unde:

$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{ L, R \}$;

Fie MT' = $(Q', \Sigma, \Gamma, \delta', q'_0, q'_a, q'_r)$ unde:

$\delta' : Q' \times \Gamma \rightarrow Q' \times \Gamma \times \{ L, R, S \}$;

$\delta'(q_i, a) = (q_j, b, S)$

$\delta (q_i, a) = (q_k, b, R)$, $\delta (q_k, b) = (q_j, b, L)$

=> Cele 2 modele sunt computational echivalente dar:

- e nevoie de cate o o stare auxiliara suplimentara;
- sunt necesare 2 tranzitii in loc de una.

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Definitia 1

O MT cu mai multe benzi este o MT standard

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ care:

(i) are $n \geq 1$ benzi de lucru si $n \geq 1$ cursoare corespunzatoare;

(ii) initial prima banda contine secventa de intrare iar celelalte n-1 benzi sunt vide;

(iii) $\delta : Q \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{ L, R, S \}^n$,

$\delta(q_i, a_1, a_2, \ldots, a_n) = (q_j, b_1, b_2, \ldots, b_n, L, R, S, L, \ldots, S)$.

Variante de masini Turing

Teorema 1

$\forall$ M = MT cu mai multe benzi =>

$\exists$ S = MT standard a.i. L(S) = L(M)

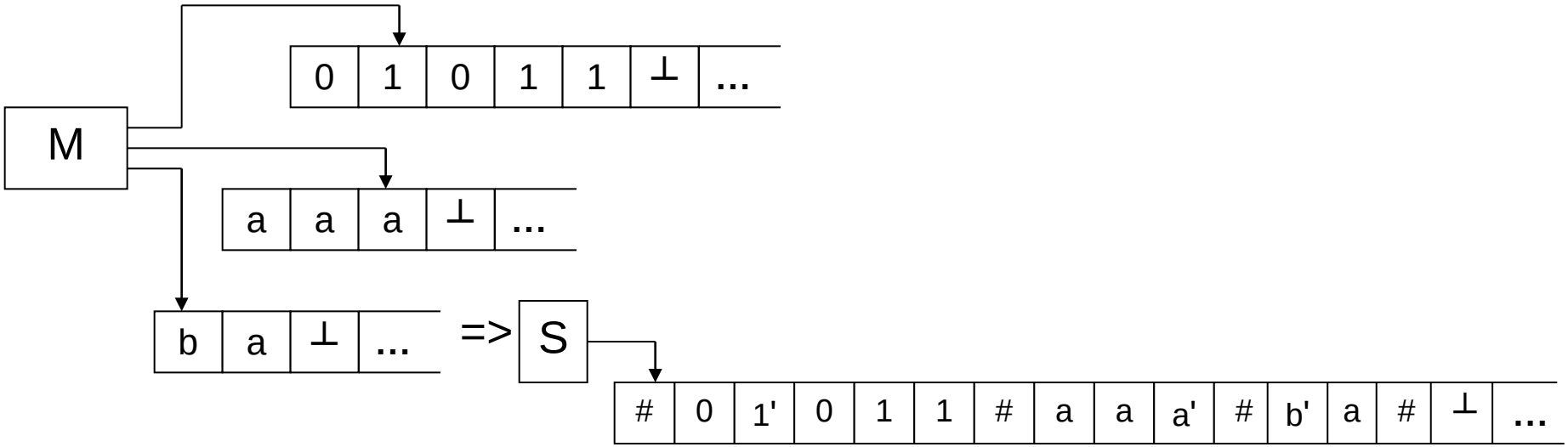
ideea demonstratiei

Fie M = ($Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r$) o MT cu $n \geq 2$ benzi si

S = ($Q', \Sigma', \Gamma', \delta', q'_0, q'_a, q'_r$) o MT standard.

Variante de masini Turing

Exemplu:



=> (i) un simbol aditional #, pt delimitator;

(ii) $\forall\ \alpha \in \Gamma \rightarrow \exists\ \alpha' \in \Gamma'$

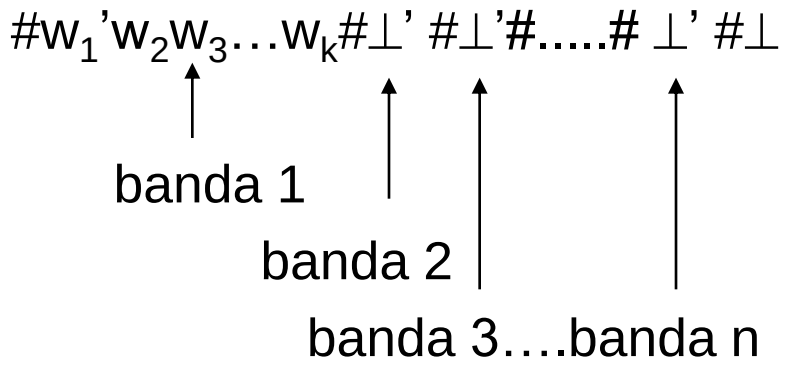
=> S devine o MT cu mai multe benzi si cursoare virtuale.

Variante de masini Turing

demonstratie

S = “Fie secventa de intrare $w=w_1w_2\ldots w_k \in \Sigma^*$:

1. S isi formateaza banda de intrare pt a reprezenta cele n benzi ale M, la momentul initial:



Variante de masini Turing

2. S scanează unica sa bandă de la primul delimitator # până la cel de-al (n+1)lea delimitator # în scopul de a determina simbolurile aflate în dreptul cursoarelor virtuale

Apoi, S mai face o parcurgere pentru a actualiza benzile virtuale conform definiţiei funcţiei de tranziţie a lui M;

3. Dacă S deplasează unul dintre cursoarele virtuale la dreapta unui delimitator #, aceasta înseamnă că M a mutat cursorul corespunzător peste porţiunea vidă a benzii respective. => S:

- depune simbolul special ⊥ în celula respectivă de pe unica sa bandă,
- deplasează conţinutul benzii sale, de la acea celulă până la cel mai din dreapta delimitator #, cu o celulă la dreapta

Apoi, continuă simularea ca mai sus.”

Variante de masini Turing

Corolar 1

$\forall L \subseteq \Sigma^*$ este Turing-recunoscut $\Leftrightarrow$

$\exists$ o MT cu mai multe benzi, M, a.i. L=L(M).

demonstraţie

"$\Rightarrow$"

$\forall$ MT standard este o MT cu mai multe benzi trivială (n = 1).

"$\Leftarrow$"

Fie $M \in MT$, M are mai multe benzi: L=L(M);

Cf. Teoremei 1, $\exists$ $S \in MT$ standard, echivalentă cu M;

$\Rightarrow$ L = L(S).

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

- MTN ≈ AFN, APDN;
- Modelul de calcul: arbore;
- MTN accepta secventa de intrare $\Leftrightarrow$

  cel putin una dintre ramurile arborelui are ca eticheta a frunzei o configuratie de acceptare.

Definitia 2

O MT nedeterminista este o MT standard

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ unde:

$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$.

Variante de masini Turing

Teorema 2

$\forall$ N = MT nedeterminista =>

$\exists$ S = MT standard a.i. L(S) = L(N)

ideea demonstratiei

Fie N o MT nedeterminsită;

este suficient să construim o MT, M, cu trei benzi care să simuleze N deoarece,

conform Teoremei 1, vom putea apoi să construim o MT standard, S, care să o simuleze pe M.

Variante de masini Turing

- M va incerca toate ramurile de calcul posibile ale N;
- daca va ajunge vreodata in starea q_a atunci M recunoaste secventa de intrare, altfel simularea nu se va termina
- Cautarea unei configuratii de acceptare efectuata de M in arborele lui N:

1. DEPTH FIRST;
2. BREADTH FIRST

cazul (1): exploram in adancime o ramura infinita si ignoram una valida….

cazul (2): sigur vom examina toate nodurile arborelui pana gasim configuratia de acceptare.

Variante de masini Turing

demonstratie

Fie $\delta_N : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$ functia de tranzitie a lui N

Notam $b=\max\{ card(A) \mid A \in \mathcal{P}(Q \times \Gamma \times \{L,R\}) = codom(\delta_N) \}$

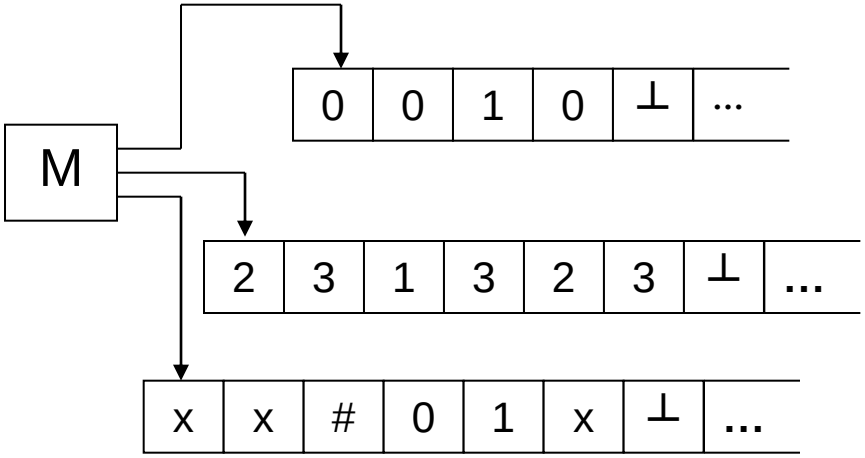
=> b=nr max de fíi pe care ii poate avea un nod din arborele de derivare al lui N

Notam cu $\Sigma_b=\{1,2,\ldots,b\}$

Construim o MT determinista M cu 3 benzi astfel:

- banda 1: conţine permanent cuvântul de intrare;
- banda 2: ţine evidenţa poziţiei din arbore în care se află M;
- banda 3: contine si actualizeaza permanent un duplicat al continutului benzii de lucru a lui **N**, reprezentand una dintre ramurile de calcul nedeterminist efectuat de **N**.

Variante de masini Turing



banda de intrare

banda de adresa

banda de simulare

Banda 2 contine in orice moment un cuvant $\alpha \in \Sigma_b^*$;
acest cuvant reprezinta:

- fie una dintre ramurile de calcul nedeterminist efectuat de N, de la radacina pana la nodul a carui adresa este exact α;
- fie o adresa nevalida.

Variante de masini Turing

M="Fie secventa de intrare w:

1. Iniţial, banda 1 conţine cuvântul de intrare w , pe banda 2 se genereaza primul cuvant $\alpha \in \Sigma_b^*$ (in ordine lexicografica) iar banda 3 este vida
2. Se copiază conţinutul benzii 1 pe banda 3
3. Se utilizează banda 3 pentru a simula una dintre ramurile calculului nedeterminist efectuat de N pe intrarea w

   Înainte de a simula un pas de calcul efectuat de N, M examineaza simbolul urmator de pe banda 2 pt a determina care dintre posibilitatile de continuare oferite de δ_N trebuie aleasa.

Variante de masini Turing

Daca nu mai sunt simboluri pe banda 2, atunci se trece la Pasul 4

Daca alegerea facuta este nevalida, atunci se trece tot la Pasul 4

Dacă s-a intrat într-o configuraţie de respingere, se merge tot la Pasul 4

Dacă s-a intrat într-o configuraţie de acceptare, M recunoaşte intrarea w şi se opreste

4. Se înlocuieşte secvenţa de pe banda 2 cu următorul cuvânt peste Σ_b, în ordine lexicografică. Se simulează următoarea ramură de calcul al N reluând de la Pasul 2.”

Variante de masini Turing

Corolar 2

$\forall L \subseteq \Sigma^*$ este Turing-recunoscut $\Leftrightarrow$

$\exists$ o MT nedeterminista, N, a.i. L=L(N)

demonstraţie

"$\Rightarrow$"

$\forall$ MT standard este o MT nedeterminista trivială

"$\Leftarrow$"

Fie $N \in MT$, N este nedeterminista: L=L(N);

Cf. Teoremei 2, $\exists$ $S \in MT$ standard, echivalentă cu N;

$\Rightarrow$ L = L(S).

Variante de masini Turing

Definitie 3

O MT nedeterminista se numeste decidenta $\Leftrightarrow$

$\forall w \in \Sigma^*$, toate ramurile ei de calcul se opresc

Corolar 3

$\forall L \subseteq \Sigma^*$ este decidabil $\Leftrightarrow$

$\exists$ o MT nedeterminista decidenta, N, a.i. L=L(N).

Variante de masini Turing

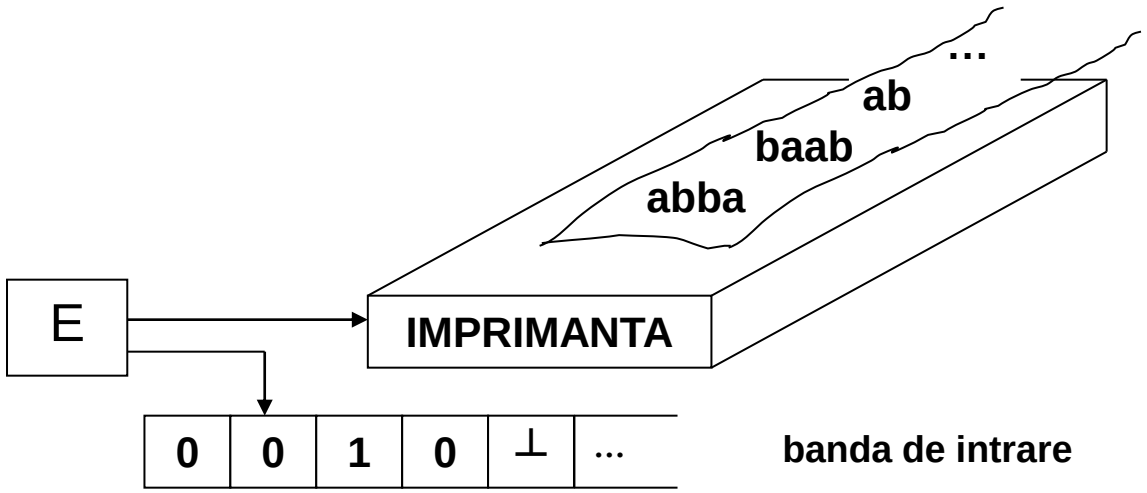
1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Limbajele Turing recunoscute = limbaje recursiv enumerabile

Enumerator

Imprimanta



Modul de lucru:

- initial: banda de intrare este vida;
- limbajul enumerat de E: multimea tuturor secventelor de simboluri printate de imprimanta;
- situatia de ciclare: tiparirea unei liste infinite de cuvinte.

Variante de masini Turing

Teorema 3

Fie $L \subseteq \Sigma^*$; L este Turing-recunoscut $\Leftrightarrow$

$\exists$ $E \in MT$ a.i. $L=L(E)$

demonstratie

"$\Leftarrow$"

Fie $E \in MT$ a.i. $L=L(E)$

M=" Fie secventa de intrare w:

1. Se ruleaza enumeratorul E pe intrarea w.
   De fiecare data cand E printeaza un cuvant, se compara acest cuvant cu secventa w.
2. Daca apare o coincidenta, atunci M accepta."

Evident, M accepta secventele din lista tiparita de E.

Variante de masini Turing

"$\Rightarrow$"

Fie $M \in MT$ a.i. $L=L(M)$.

E=" Indiferent care este cuvantul de pe banda de intrare a lui E, acesta este ignorat:

1. Pentru fiecare i=1,2,… se repeta urmatorii pasi:

2. M executa **i** pasi de calcul pentru fiecare cuvant de intrare $s_1,s_2,\ldots,s_i \in \Sigma^*$.

3. Daca vreunul dintre cei **i** pasi de calcul aduce M intr-o configuratie de acceptare, atunci cuvintele corespunzatoare din sirul $s_1,s_2,\ldots,s_i$ sunt tiparite de imprimanta lui E."

Variante de masini Turing

Observatii

1. Pentru a construi E a fost nevoie se o listare a cuvintelor din Σ: orice listare (fara repetitii) este acceptabila.
2. Evident, daca M accepta un cuvant oarecare $s \in \Sigma^*$, acesta apare la un moment dat in lista tiparita de E. De fapt, el va fi tiparit de o infinitate de ori deoarece M reia calculul de la pasul 1 pt fiecare **i**, oridecateori lista $s_1, s_2, \ldots, s_i$ se lungeste cu inca un cuvant.
3. Procedura de mai sus simuleaza rularea lui M in paralel pe toate cuvintele de intrare posibile:

Variante de masini Turing

i=1	s_1	1 pas de calcul	
i=2	s_1	2 pasi de calcul	
	s_2	2 pasi de calcul	
i=3	s_1	3 pasi de calcul	
	s_2	3 pasi de calcul	acceptat
	s_3	3 pasi de calcul	
i=4	**s_1**	4 pasi de calcul	acceptat
	s_2	4 pasi de calcul	acceptat
	s_3	4 pasi de calcul	
	s_4	4 pasi de calcul	acceptat

.....

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Variante de MT

- banda infinita la dreapta si in jos;
- *RAM* (*Random Access Machines)*
- cursor capabil sa examineze blocuri de celule;
- cursor capabil fie sa modifice simbolul citit fie sa se deplaseze;
- etc.

Variante de masini Turing

Variante de MT:

- **MT cu 1! banda infinita in ambele directii**:

$\not b$	$\ldots$	a_{-2}	a_{-1}	a_0	a_1	a_2	$\ldots$	a_n	$\not b$

simulata cu o MT cu 2 benzi.

a_0	a_1	a_2	$\ldots$
$\#$	a_{-1}	a_{-2}	$\ldots$

Variante de masini Turing

Variante de MT:

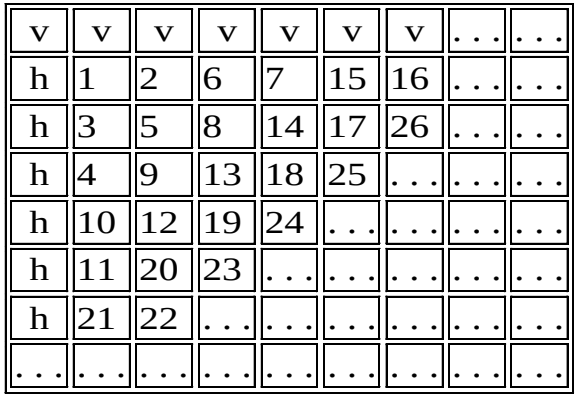
- **MT bidimensionala =**

o MT care are:

- ✓ o unitate centrala
- ✓ un cap de citire/scriere
- ✓ o banda de lucru
  - ✓ bidimensionala
  - ✓ infinita la drepata şi in jos
  - ✓ impartita pe linii şi coloane, respectiv in celule (locatii).

v	v	v	v	v	v	v	. . .	. . .
h	1	2	6	7	15	16	. . .	. . .
h	3	5	8	14	17	26	. . .	. . .
h	4	9	13	18	25	. . .	. . .	. . .
h	10	12	19	24	. . .	. . .	. . .	. . .
h	11	20	23	. . .	. . .	. . .	. . .	. . .
h	21	22	. . .	. . .	. . .	. . .	. . .	. . .
. . .	. . .	. . .	. . .	. . .	. . .	. . .	. . .	. . .

Variante de masini Turing



MT bidimensionale (cont.)

Definitie

MT^2 = un sistem $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$ unde:

Q = mulţime finită, nevida: mulţimea stărilor;

Σ = mulţime finită, nevida: alfabetul de intrare; $\Sigma \cap Q = \emptyset$;

Γ = mulţime finită, nevida: alfabetul benzii; $\Sigma \subset \Gamma$, $\sqcup, h, v \in \Gamma \setminus \Sigma$

(h / v marcheaza limita stanga / superioara a benzii bidimensionale);

$\delta : Q \times \Gamma \to Q \times \Gamma \times \{ L , R \}$: funcţia de tranziţie;

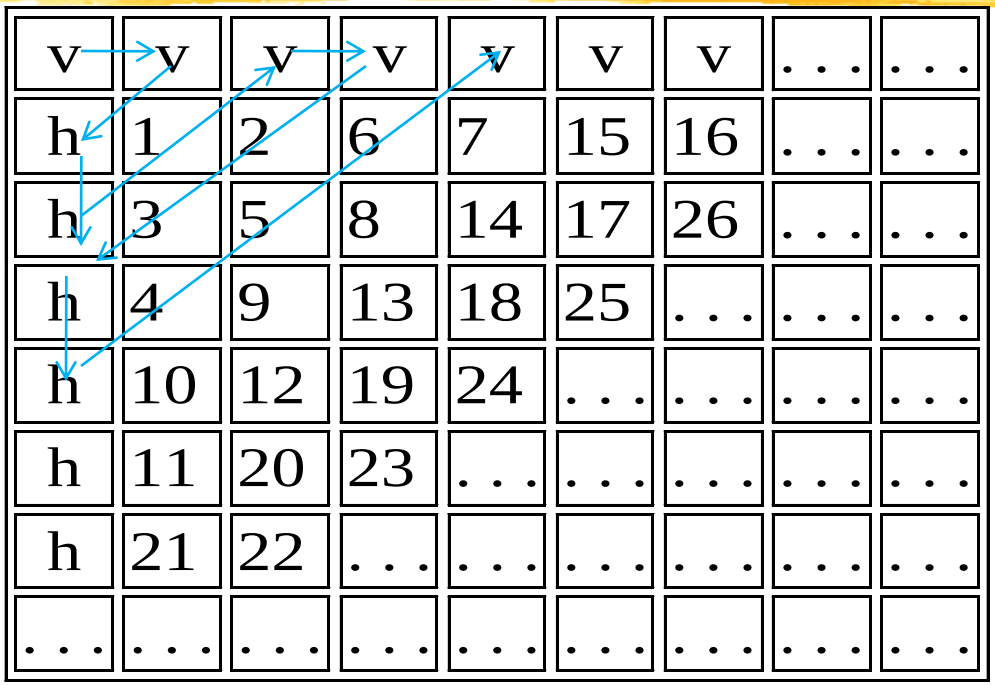
$q_0 \in Q$: starea iniţială;

$q_a \in Q$: starea finala de acceptare a secventei de intrare;

$q_r \in Q$: starea finala de respingere a secventei de intrare.

Variante de masini Turing

MT bidimensionale (cont.)


v v v v v v v . . . . . .
h 1 2 6 7 15 16 . . . . . .
h 3 5 8 14 17 26 . . . . . .
h 4 9 13 18 25 . . . .
h 10 12 19 24 . . . .
h 11 20 23 . . . .
h 21 22 . . . .
. . . .

Variante de masini Turing

Variante de MT:

- ***Random Access Machines***

un model idealizat al calculatorului v.Neumann:

- ✓ are:
  - un numar finit de registri,
  - un numar infinit de locatii de memorie;
- ✓ poate “stoca” in memorie un “program”;
- ✓ initial:
  - “programul” este stocat intr-un bloc de locatii de memorie adiacente, incepand de la locatia de adresa 1,
  - toti registrii şi toate celelelate locatii de memorie sunt setate la 0.

Variante de masini Turing

Random Access Machines *(cont.)*

Setul de instructiuni:

- ✓ *loadc X, I*

  => [X] ← I (se depune in registrul X constanta I)

- ✓ *load X, M*

  => [X] ← M (se depune in registrul X continutul locatiei de memorie M)

- ✓ *loadI X, M*

  => [X] ← M (se depune in registrul X continutul locatiei de memorie la care trimite pointerul din locatia M)

- ✓ *store X, M*

  => M ← [X] (se depune in locatia de memorie M continutul registrului X) ->

Variante de masini Turing

Random Access Machines *(cont.)*

Setul de instructiuni:

- ✓ *add X, Y*

  => [X] ← [X] + [Y]

- ✓ *jmp M*

  => salt la instructiunea din locatia M

- ✓ *jmz X, M*

  => daca [X]=0 atunci salt la instructiunea din locatia M

- ✓ *halt*

  ⇒calculul se opreste.

Variante de masini Turing

Random Access Machines *(cont.)*

Teorema

RAM ⇔ MT

Demonstratie (indicatii)

(i) Pt a simula calculul facut de o RAM, o MT (cu mai multe benzi), trebuie sa memoreze continutul registrilor şi memoriei acesteia astfel:

B_1 = banda contorului instructiunilor =

= stocheaza adresa locatiei de memorie in care este stocata urmatoarea instructiune de executat; initial este 1;

B_2 = banda adreselor de memorie =

= stocheaza adresa locatiei de memorie in care se va executa o operatie de tip *load* sau *store*; -->

Variante de masini Turing

Random Access Machines *(cont.)*

B_3 = banda registrilor =

= stocheaza continutul tuturor registrilor astfel:

<codul binar al registrului>*<codul binar al continutului registrului>

….

ex.: fie 3 registri, R1, R2, R3 cu valorile respective 10, 20, 30;

atunci B_3 contine: # 1*1010 # 10*10100 # 11*11110 #

B_4 = banda memoriei =

= analog cu B_3, memoreaza # <adresa>*<continut> #

pentru a stoca o instructiune se memoreaza mnemonicul urmat de argumente;

B_5, B_6,B_7 = banzi de lucru =

= simuleaza calculul efectuat de RAM.

Variante de masini Turing

Random Access Machines *(cont.)*

(ii) simularea calculului facut de RAM

TM incepe simularea programul stocat in memorie şi cu B_1 (banda contorului instructiunilor) initializata cu 1; fiecare pas de calcul efectuat de RAM este simulat de MT in maimulti pasi şi folosind toate ceele 7 benzi:

P1: citeste B_1 = banda contorului instructiunilor

P2: cauta instructiunea curenta in memorie

P3: memoreaza in mnemonicul instructiunii şi adresa registrului (care contine argumentul) in unitatea sa centrala. Stocheaza adresa argumentului pe B_4 = banda memoriei

P4: extrage valorile continute in B_3 = banda registrilor şi/sau B_4 = banda memoriei şi se depun intr-o banda de lucru

P5: executa operatia cu ajutorul benzilor de lucru

P6: actualizeaza B_1, B_3, B_4.

Variante de masini Turing

Random Access Machines *(cont.)*

Exemplu: simularea unei adunari

pp. B_1 = banda contorului instructiunilor contine 101 (i.e. urmatoarea instructiune de executat se afla la adresa 101) =>

MT scaneaza B_4 = banda memoriei in cautarea valorii #101 (adica a locatiei unde se afla instructiunea care trebuie executata)

pp. B_4 contine …#101*<add>,11,110# (deci RAM trebuia sa adune continutul registrilor 11 şi 110)

MT memoreaza in unitatea sa centrala ”add”, 11, 110 i.e. trece in starea $q_{add,11,110}$ care codifica actiunea MT de a aduna continutul registrilor 11 şi 110 şi de a depune rezultatul in registrul 11 →

Variante de masini Turing

Random Access Machines *(cont.)*

MT scaneaza B_3 = banda registrilor in cautarea valorii #11*
pp. B_3 contine …#11*10110#... (i.e. continutul registrului 11 este 10110)

MT copiaza pe una din benzile de lucru valoarea 10110

MT scaneaza B_3 = banda registrilor in cautarea valorii #11* şi inlocuieste valoarea 10110 din registrul 11 cu rezultatul calculului efectuat pe banda de lucru (aceasta poate implica deplasarea continutului B_3 la dreapta/stanga)

Se aduna 1 la continutul B_1 = banda contorului instructiunilor .

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterministe
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Am definit mai multe variante de **MT** şi am demonstrat echivalenţa lor .

Există şi alte modele de calculabilitate:

- funcţiile λ-calculabile: Alonzo CHURCH, Stephen Cole KLEENE,1934;
- funcţiile general recursive: Kurt Godel, 1935
- sistemele (masinile) Post: Emil POST, 1936
- algoritmii normali Markov: A.A.MARKOV, 1954.

Toate au aceeaşi caracteristică: acces nerestricţionat la o memorie nelimitată.

În plus, toate sunt echivalente cu **MT** (şi deci unele cu altele)

=> clasa algoritmilor pe care o descriu este unică şi naturală.

vezi si clasa limbajelor de programare.

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterminsite
5. Enumeratoare
6. Alte variante de MT
7. Echivalenta modelelor de calculabilitate
8. Problema a 10-a a lui Hilbert

Variante de masini Turing

Algoritm: o notiune primara

Problema a 10-a a lui Hilbert.

Variante de masini Turing

Congresul Internaţional al matematicienilor, 1900, Paris;

Conferinta lui David HILBERT: lista celor 23 probleme

să se găsească un algoritm care să verifice dacă un polinom în oricâte variabile admite o rădăcină în $\mathbf{Z}$.

Observatii

- "… un proces cu ajutorul căruia, după un număr finit de operaţii, să se determine …."
- Hilbert presupunea că algoritmul există si trebuie doar descoperit
- Lipsa unei definitii formale pentru notiunea de algoritm → solutii pentru cazuri particulare ale problemelor dar nu pentru o clasa intreaga.

Variante de masini Turing

Teza Church-Turing

Clasa algoritmilor (functiilor intuitiv calculabile) coincide cu clasa functiilor λ-calculabile (functiilor calculabile cu MT).

Problema a 10a a lui Hilbert

1970, Yuri MATIJASEVIČ, (Martin DAVIS, Hilary PUTNAM, Julia ROBINSON):

nu există nici un algoritm care să verifice dacă un polinom oarecare are radacini intregi $\Leftrightarrow$

Problema a 10a a lui Hilbert este nerezolvabilă algoritmic.

Variante de masini Turing

Fie D = {p | p este un polinom care admite cel putin o radacina in Z}

D este decidabil ?

D este Turing–acceptat dar nu este decidabil

<u>Cazul unar:</u>

Fie D_1={p_1|p_1 este un polinom intr-o singura variabila x, care admite cel putin o radacina intreaga}

Construim o MT, M_1, care accepta D_1:

M_1 = “Fie p_1 un polinom oarecare in variabila x:

1. Se evalueaza p_1 succesiv pentru x=0, x=1, x=-1, x=2, x=-2, x=3, …,
2. Daca la un moment oarecare se obtine 0, atunci M_1 accepta p_1.”

Variante de masini Turing

Cazul general:

putem construi o MT, M, similara care va testa fiecare polinom de intrare p pentru diferite combinatii de valori, in functie de numarul de variabile

n=2: $(x_1,x_2)\in\{(0,0), (0,1), (1,0), (1,1), (0,2), (2,0), (1,2), (2,1), (2,2), (0,3), (3,0),\ldots\}$;

n=3: $(x_1,x_2,x_3)\in\{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1),\ldots.$

Observam ca: M_1 si M accepta limbajele D_1, respectiv D, dar nu decid asupra lor.

Variante de masini Turing

Teorema 4

Daca un polinom p de o singura variabila admite radacini intregi,

acestea se afla in intervalul $\left[-k \cdot \frac{|c_{max}|}{|c_1|}, k \cdot \frac{|c_{max}|}{|c_1|}\right]$

unde: k=nr de termeni (monoame) din p;

c_{max} = coeficientul cel mai mare in valoare absoluta

c_1= coeficientul termenului de grad maxim.

Variante de masini Turing

=> M_{11} = “Fie p_1 un polinom oarecare in variabila x:

1. Se calculeaza $t_2= k*|c_{max}/c_1|$, $t_1=-t_2$ si $z=[t_2]$.
2. Se evalueaza p_1 pentru z.
3. Daca $p_1(z)=0$, atunci M_{11} accepta; altfel, z:=z-1.
4. Daca $z<t_1$ atunci M_{11} respinge; altfel, reia de la P2.”

Teorema lui MATIJASEVIČ

Variante de masini Turing

1. Terminologia de descriere a MT; codificari; exemple
2. MT cu 3 “deplasari”
3. MT cu mai multe benzi
4. MT nedeterminsite
5. Enumeratoare
6. Echivalenta modelelor de calculabilitate
7. Problema a 10-a a lui Hilbert

